Tarea 5.1: IA con plataforma AWS

Mini-Veo-Basket-AMT

Programación de Inteligencia Artificial

Alberto Maldonado Triana



índice

1.- Introducción..... 3

2.- Arquitectura del sistema..... 3
2.1- Arquitectura del sistema.....3
2.2- Módulos de la arquitectura.....4
2.3- Funcionamiento de Amazon Rekognition..... 4

3.- Tecnologías utilizadas.....4

4.- Flujo de trabajo..... 5

5.- Ejemplo práctico de uso..... 5

6.- Instalación y configuración para la app..... 6

7.- Buenas prácticas..... 7

8.- Limitaciones del sistema..... 8

9.- Conclusión..... 8

1.- Introducción

- Mini-Veo-Basket-AMT es una aplicación inspirada en VeoCam, pero a nivel muy básico, y sin conseguir estadísticas de los jugadores en tiempo real, funciona subiendo una foto de una jugada de baloncesto y la aplicación:

    1. Detecta a los jugadores y el balón con Amazon Rekognition.

    2. Dibuja con OpenCV una caja verde alrededor de cada jugador y una caja naranja alrededor del balón haciendo uso de sus etiquetas.

    3. Dibuja una línea desde cada jugador hasta el balón con la distancia en píxeles de la foto suba, y marca en rojo al jugador más cercano a el balón.

    4. Le pasa la imagen a Gemini haciendo uso de su APIkey, que escribe consejos de entrenador para conseguir canasta a partir de la jugada mostrada en la foto.

    5. Lo muestra todo en una página web hecha con Gradio.

2.- Arquitectura del sistema

La aplicación Mini-Veo-Basket-AMT está desarrollada en Python puramente en la que cada archivo se encarga de una funcionalidad concreta del sistema, es decir no todo está metido en un fichero único.

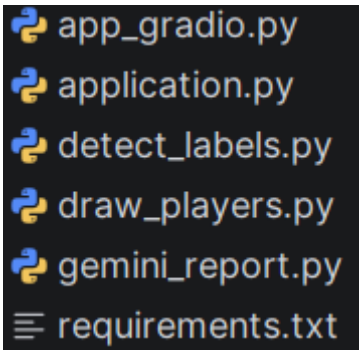
El proyecto está dividido en diferentes módulos que trabajan conjuntamente para recibir una imagen, analizarla mediante inteligencia artificial, procesar la foto y mostrar los resultados al usuario en la web.

2.1- Arquitectura del sistema

El proyecto contiene los siguientes ficheros:

- application.py: es el archivo principal de la aplicación. Contiene el servidor Flask, recibe las peticiones POST en formato JSON, gestiona la comunicación con los buckets de Amazon S3 y hace las llamadas a las partes de el proyecto.
- detect_labels.py: es el encargado de comunicarse con Amazon Rekognition mediante la funcionalidad DetectLabels, para detectar los jugadores y el balón presentes en la fotografía.
- draw_players.py: Procesa la imagen mediante OpenCV y se encarga de dibujar los recuadros alrededor de los jugadores y el balón con sus diferentes colores, trazar las líneas de los pases y calcular las distancias en píxeles e identificar al jugador más cercano al balón.

- gemini_report.py: Se comunica con Google Gemini mediante su API, enviándole la imagen para obtener recomendaciones tácticas sobre la jugada en forma de entrenador.
- app_gradio.py: Hace la interfaz web desarrollada con Gradio, desde la que el usuario puede subir fotografías, iniciar el análisis o reiniciar y ver los resultados.
- requirements.txt: contiene las librerías y dependencias necesarias para lanzar el proyecto.



2.2- Módulos de la arquitectura

- Comunicación: peticiones HTTP POST con JSON entre Gradio y Flask.
- Servicios web: Flask para la API y Gradio para la interfaz.
- Servicios de IA: Amazon Rekognition y Google Gemini.
- Tareas: procesamiento de imágenes con OpenCV.
- Almacenamiento: Amazon S3 mediante dos buckets, origen y destino.

2.3- Funcionamiento de Amazon Rekognition

1. Se envía la foto de la jugada a Rekognition.
2. Se utiliza la funcionalidad DetectLabels de la app.
3. Analiza la imagen y mandas las coordenadas donde haya un jugador.
4. La aplicación utiliza esos datos para dibujar los recuadros mediante OpenCV.

3.- Tecnologías utilizadas

Para hacer la app se han utilizado las siguientes tecnologías:

- **Python:** Como lenguaje de programación principal.
- **Amazon AWS:** La plataforma de servicios en la nube.
- **Amazon Rekognition:** Para la detección de jugadores y el balón.
- **Amazon S3:** Para almacenamiento de las imágenes originales y las procesadas.
- **OpenCV:** Para el procesamiento y anotación de imágenes de los recuadros.
- **Google Gemini:** Genera los consejos tácticos mediante la IA.
- **Flask:** Crea la API que recibe peticiones POST.
- **Gradio:** Genera la interfaz web.

4.- Flujo de trabajo

El funcionamiento de la aplicación sigue una serie de pasos desde que el usuario sube una fotografía hasta que recibe los resultados:

1. El usuario sube una fotografía de baloncesto desde la interfaz de Gradio.
2. La imagen se guarda en el bucket de origen de Amazon S3.
3. Gradio envía una petición POST con formato JSON al servidor Flask (application.py).
4. Flask llama a Amazon Rekognition para detectar los jugadores y el balón.
5. Los resultados se envían a OpenCV, que dibuja los recuadros, las líneas y las distancias.
6. La imagen procesada se guarda en el bucket de destino de Amazon S3.
7. Se envía la imagen a Google Gemini para generar recomendaciones tácticas.
8. Finalmente, Gradio muestra la imagen analizada y el informe generado.

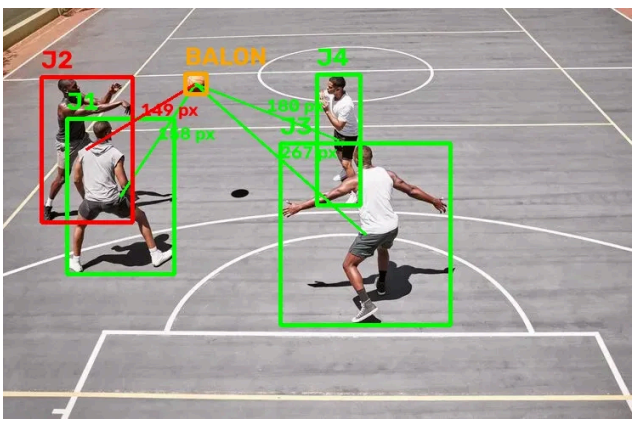
5.- Ejemplo práctico de uso

Para comprobar el funcionamiento de la app, se realiza un análisis utilizando una fotografía de una jugada de baloncesto.

Como por ejemplo:



1. Se accede a la interfaz web de Gradio desde http://127.0.0.1:7860.
2. Se analiza la jugada de la foto.
3. Se pulsa el botón «Analizar jugada» para iniciar el análisis.
4. La aplicación detecta los jugadores y el balón, y procesa la imagen ya con las etiquetas puestas.
5. Se muestra la fotografía con los recuadros, las líneas y las distancias calculadas, una vez que termina en análisis, como por ejemplo:



6. Gemini genera un informe con recomendaciones tácticas.
7. El usuario puede descargar la imagen resultante o pulsar «Nueva consulta» para realizar otro análisis.

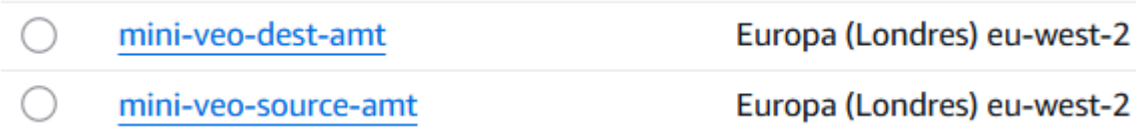
Resultado y finalidad de la app: Obtener una imagen anotada y un informe de recomendaciones que ayudan a conseguir esa canasta a partir de ese momento de la foto.

6.- Instalación y configuración para la app

Para ejecutar la aplicación es necesario disponer de Python 3, una cuenta de AWS (es posible con una cuenta gratuita) y una clave API de Google Gemini (también es posible con cuenta gratuita).

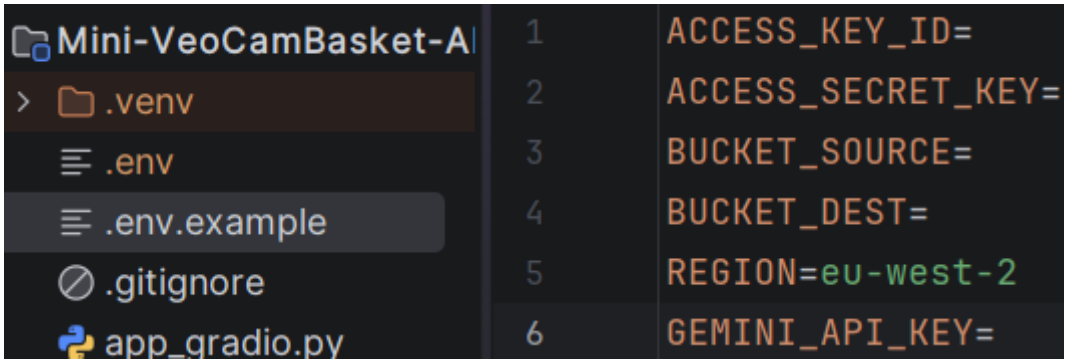
Los pasos para ponerla en marcha son:

➢ Crear dos buckets en Amazon S3, uno de origen y otro de destino.

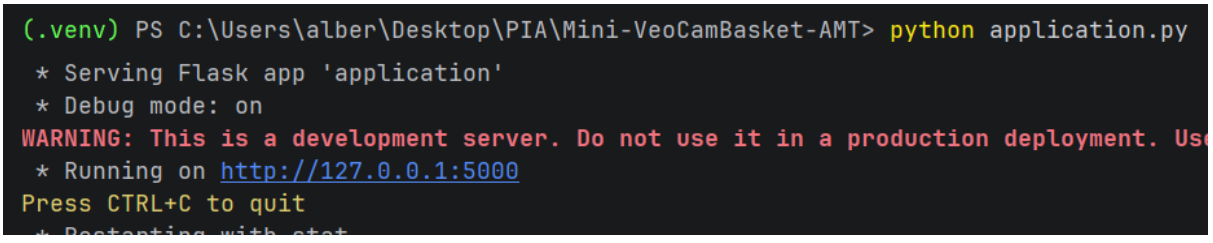


➢ Descargar el proyecto e instalar las dependencias mediante pip install -r requirements.txt.

➢ Crear el fichero .env a partir de .env.example e introducir las credenciales de AWS, los nombres de los buckets, la región y la clave API de Gemini.



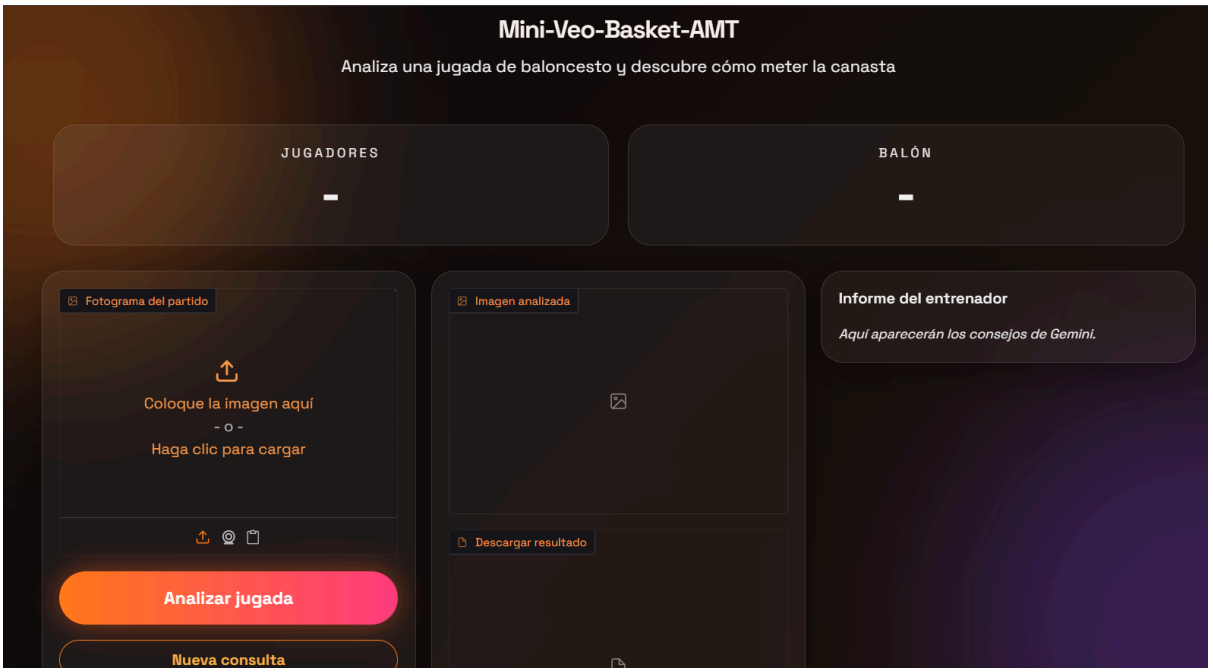
➢ Iniciar el servidor Flask ejecutando python application.py.



➢ En otra terminal, iniciar Gradio con python app_gradio.py y en un entorno virtual.

```
(.venv) PS C:\Users\alber\Desktop\PIA\Mini-VeoCamBasket-AMT> python app_gradio.py
* Running on local URL:  http://127.0.0.1:7860
* To create a public link, set `share=True` in `launch()`.
```

➢ Acceder a http://127.0.0.1:7860 para utilizar la aplicación una vez arrancado los dos ficheros.



7.- Buenas prácticas

Durante el desarrollo se han aplicado diferentes buenas prácticas:

- **Separación de funcionalidades de cada fichero:** Cada archivo tiene una responsabilidad en concreto.
- **requirements.txt:** Que permite instalar todas las dependencias necesarias.
- **Variables de entorno:** las credenciales se almacenan en un fichero .env y así no es visible en todo el proyecto.
- **Escalabilidad:** Es facil el mantenimiento y la modificación del proyecto.

8.- Limitaciones del sistema

La aplicación presenta algunas limitaciones/dificultades que hay que tener en cuenta:

- Amazon Rekognition detecta personas y objetos, pero no comprende específicamente el baloncesto, ni de los jugadores, es decir si hay periodistas o algo asi, los detecta como jugador
- Puede identificar árbitros o espectadores como jugadores.
- Las distancias calculadas están expresadas en píxeles y no en metros reales.
- El jugador más cercano al balón no tiene por qué ser quien lo posee.
- Las recomendaciones de Gemini son interpretaciones de la jugada y pueden contener errores.

9.- Conclusión

El desarrollo de Mini-Veo-Basket-AMT es una app que combina diferentes tecnologías de inteligencia artificial y servicios en la nube.

Mediante el servicio de Amazon Rekognition se detectan jugadores y balones, OpenCV permite procesar visualmente la fotografía y Gemini genera recomendaciones tácticas.

Además, el uso de Flask, Gradio y Amazon S3 permite integrar todos estos servicios en una aplicación funcional.

En conclusión, el proyecto intenta recrear una funcionalidad de una cámara profesional de reconocimiento en el deporte, haciendo uso de las técnologias dadas en clase.